

CMake 教程

参考内容：

- <https://subingwen.cn/cmake/CMake-primer/>
- <https://subingwen.cn/cmake/CMake-advanced/>

本文整理了CMake 相关知识

目录

- 1 CMake 概述
- 2 CMake 基础使用
 - 2.1 注释
 - 2.2 最小 CMakeLists.txt
 - 2.3 执行 CMake
 - 2.4 设置 C++ 标准
 - 2.5 变量
 - 2.6 搜索源文件
 - 2.7 包含头文件目录
 - 2.8 生成可执行程序
 - 2.9 生成库文件
 - 2.10 指定输出目录
 - 2.11 链接库文件
 - 2.12 日志输出
 - 2.13 变量和列表操作
 - 2.14 宏定义
 - 2.15 常用预定义变量
- 3 嵌套 CMake 工程
 - 3.1 嵌套工程的目录关系
 - 3.2 add_subdirectory
 - 3.3 多目录工程示例
- 4 CMake 流程控制
 - 4.1 条件判断 if

- 4.2 基本表达式
 - 4.3 逻辑判断
 - 4.4 数值比较
 - 4.5 字符串比较
 - 4.6 文件和路径判断
 - 4.7 列表和路径比较
 - 4.8 循环 foreach
 - 4.9 循环 while
 - 5 综合示例
-

1 CMake 概述

CMake 是一个跨平台的项目构建工具。它本身通常不直接完成编译，而是根据 `CMakeLists.txt` 中描述的规则，生成本地构建系统需要的文件。例如在 Linux 下生成 `Makefile`，在 Ninja 环境下生成 Ninja 构建文件，在 Visual Studio 环境下生成对应工程文件。

它解决的问题主要有：

- 不同平台的构建方式不同，手写 `Makefile` 可移植性差；
- 项目文件多时，手工维护编译命令和依赖关系容易出错；
- 大型工程需要管理可执行程序、静态库、动态库、头文件目录和第三方库；
- 同一套工程希望在 Linux、Windows、macOS 或 IDE 中构建。

CMake 的常见工作流程如下：

1. 编写 `CMakeLists.txt`；
2. 执行 `cmake` 生成本地构建系统；
3. 执行 `cmake --build` 或 `make` 完成编译；
4. 得到可执行程序或库文件。

推荐使用“源码目录”和“构建目录”分离的方式，也就是 out-of-source build。这样 CMake 生成的中间文件不会污染源码目录。

```
mkdir build
cd build
cmake ..
cmake --build .
```

2 CMake 基础使用

2.1 注释

CMake 支持行注释和块注释。

行注释使用 `#`：

```
# 这是一个行注释
cmake_minimum_required(VERSION 3.10)
```

块注释使用 `#[[ ... ]]`：

```
#[[
这里是块注释。
可以写多行内容。
]]
project(Demo)
```

2.2 最小 CMakeLists.txt

一个最小的 C/C++ 工程通常包含三个核心命令：

```
cmake_minimum_required(VERSION 3.10)
project(MyApp)
add_executable(app main.cpp)
```

含义如下：

- `cmake_minimum_required(VERSION 3.10)`：指定最低 CMake 版本；

- `project(MyApp)`：定义工程名；
- `add_executable(app main.cpp)`：用 `main.cpp` 生成名为 `app` 的可执行程序。

注意：`project()` 中的工程名和 `add_executable()` 中的可执行程序名没有必然关系。

如果有多个源文件，可以直接列出来：

```
add_executable(app main.cpp add.cpp sub.cpp mult.cpp div.cpp)
```

也可以用分号分隔：

```
add_executable(app main.cpp;add.cpp;sub.cpp;mult.cpp;div.cpp)
```

2.3 执行 CMake

假设目录结构如下：

```
calc
├─ CMakeLists.txt
├─ add.cpp
├─ div.cpp
├─ main.cpp
├─ mult.cpp
└─ sub.cpp
```

推荐这样构建：

```
cd calc
mkdir build
cd build
cmake ..
cmake --build .
```

如果使用 Makefile 生成器，也可以：

```
cmake ..  
make
```

构建完成后，会在构建目录中生成目标文件、中间文件、缓存文件和最终可执行程序。

2.4 设置 C++ 标准

可以通过变量设置 C++ 标准：

```
cmake_minimum_required(VERSION 3.10)  
project(MyApp)  
  
set(CMAKE_CXX_STANDARD 17)  
set(CMAKE_CXX_STANDARD_REQUIRED ON)  
  
add_executable(app main.cpp)
```

常见取值：

```
set(CMAKE_CXX_STANDARD 11)  
set(CMAKE_CXX_STANDARD 14)  
set(CMAKE_CXX_STANDARD 17)  
set(CMAKE_CXX_STANDARD 20)
```

`CMAKE_CXX_STANDARD_REQUIRED ON` 表示必须使用指定标准，不允许编译器退回到更低标准。

2.5 变量

CMake 使用 `set()` 定义变量，使用 `${变量名}` 取变量值。

```
set(APP_NAME app)  
set(SRC_LIST main.cpp add.cpp sub.cpp)  
  
add_executable(${APP_NAME} ${SRC_LIST})
```

变量可以保存普通字符串、路径、文件列表。

```
set(ROOT_PATH ${CMAKE_CURRENT_SOURCE_DIR})
set(INC_PATH ${ROOT_PATH}/include)
set(SRC_PATH ${ROOT_PATH}/src)
```

变量的好处是减少重复书写，提高维护性。例如目标名称、输出目录、头文件目录和库名称都可以统一放到变量中。

2.6 搜索源文件

当源文件较多时，逐个写入 `add_executable()` 或 `add_library()` 会比较麻烦。CMake 提供了搜索源文件的方式。

2.6.1 aux_source_directory

`aux_source_directory()` 可以搜索指定目录下的源文件，并存入变量。

```
aux_source_directory(<dir> <variable>)
```

示例：

```
cmake_minimum_required(VERSION 3.10)
project(Calc)

aux_source_directory(${CMAKE_CURRENT_SOURCE_DIR}/src SRC_LIST)
add_executable(app ${SRC_LIST})
```

其中：

- `<dir>`：要搜索的目录；
- `<variable>`：保存搜索结果的变量。

2.6.2 file(GLOB)

`file(GLOB ...)` 可以按通配符搜索文件。

```
file(GLOB SRC_LIST "${CMAKE_CURRENT_SOURCE_DIR}/src/*.cpp")
file(GLOB HEAD_LIST "${CMAKE_CURRENT_SOURCE_DIR}/include/*.h")
```

递归搜索使用 `GLOB_RECURSE`：

```
file(GLOB_RECURSE SRC_LIST "${CMAKE_CURRENT_SOURCE_DIR}/src/*.cpp")
```

示例：

```
cmake_minimum_required(VERSION 3.10)
project(Calc)

file(GLOB SRC_LIST "${CMAKE_CURRENT_SOURCE_DIR}/src/*.cpp")
add_executable(app ${SRC_LIST})
```

注意：`file(GLOB)` 很方便，但新加源文件后，某些情况下需要重新运行 CMake 才能更新构建系统。大型工程中也常见显式列出源文件的做法。

2.7 包含头文件目录

编译源文件时，如果代码中包含自定义头文件，需要把头文件所在目录告诉编译器。

旧式写法：

```
include_directories("${PROJECT_SOURCE_DIR}/include")
```

完整示例：

```
cmake_minimum_required(VERSION 3.10)
project(Calc)

set(CMAKE_CXX_STANDARD 17)
include_directories("${PROJECT_SOURCE_DIR}/include")

file(GLOB SRC_LIST "${CMAKE_CURRENT_SOURCE_DIR}/src/*.cpp")
add_executable(app ${SRC_LIST})
```

对应目录结构：

```
calc
├── CMakeLists.txt
├── include
│   └── head.h
└── src
    ├── add.cpp
    ├── div.cpp
    ├── main.cpp
    ├── mult.cpp
    └── sub.cpp
```

现代 CMake 更推荐对具体目标设置头文件目录：

```
add_executable(app ${SRC_LIST})
target_include_directories(app PRIVATE "${PROJECT_SOURCE_DIR}/include")
```

这两种写法都能完成头文件包含；前者影响当前目录及后续目标，后者作用范围更明确。

2.8 生成可执行程序

生成可执行程序使用 `add_executable()`：

```
add_executable(可执行程序名 源文件1 源文件2 ...)
```

示例：

```
add_executable(app main.cpp add.cpp sub.cpp)
```

也可以配合变量：

```
set(APP_NAME app)
set(SRC_LIST main.cpp add.cpp sub.cpp)

add_executable(${APP_NAME} ${SRC_LIST})
```

2.9 生成库文件

CMake 使用 `add_library()` 生成库。

2.9.1 静态库

生成静态库使用 `STATIC`：

```
add_library(calc STATIC add.cpp sub.cpp mult.cpp div.cpp)
```

在 Linux 下，生成的文件名通常是：

```
libcalc.a
```

其中 `lib` 前缀和 `.a` 后缀由系统和构建规则自动处理，CMake 中只需要写库名 `calc`。

完整示例：

```
cmake_minimum_required(VERSION 3.10)
project(CalcLib)

include_directories("${PROJECT_SOURCE_DIR}/include")
file(GLOB SRC_LIST "${CMAKE_CURRENT_SOURCE_DIR}/src/*.cpp")

add_library(calc STATIC ${SRC_LIST})
```

2.9.2 动态库

生成动态库使用 `SHARED`：

```
add_library(calc SHARED add.cpp sub.cpp mult.cpp div.cpp)
```

在 Linux 下，生成的文件名通常是：

```
libcalc.so
```

完整示例：

```
cmake_minimum_required(VERSION 3.10)
project(CalcLib)

include_directories("${PROJECT_SOURCE_DIR}/include")
file(GLOB SRC_LIST "${CMAKE_CURRENT_SOURCE_DIR}/src/*.cpp")

add_library(calc SHARED ${SRC_LIST})
```

2.10 指定输出目录

可以指定可执行程序 and 库文件的输出路径。

2.10.1 可执行程序输出路径

```
set(EXECUTABLE_OUTPUT_PATH "${PROJECT_SOURCE_DIR}/bin")
```

示例：

```
cmake_minimum_required(VERSION 3.10)
project(Calc)

set(EXECUTABLE_OUTPUT_PATH "${PROJECT_SOURCE_DIR}/bin")
file(GLOB SRC_LIST "${CMAKE_CURRENT_SOURCE_DIR}/src/*.cpp")

add_executable(app ${SRC_LIST})
```

2.10.2 库文件输出路径

```
set(LIBRARY_OUTPUT_PATH "${PROJECT_SOURCE_DIR}/lib")
```

示例：

```
cmake_minimum_required(VERSION 3.10)
project(CalcLib)

set(LIBRARY_OUTPUT_PATH "${PROJECT_SOURCE_DIR}/lib")
file(GLOB SRC_LIST "${CMAKE_CURRENT_SOURCE_DIR}/src/*.cpp")

add_library(calc STATIC ${SRC_LIST})
```

`LIBRARY_OUTPUT_PATH` 可用于设置库文件输出目录。实际项目中也常使用目标属性进行更细粒度控制。

更现代、也更精确的写法是使用目标属性：

```
add_library(calc STATIC ${SRC_LIST})

set_target_properties(calc PROPERTIES
    ARCHIVE_OUTPUT_DIRECTORY "${PROJECT_SOURCE_DIR}/lib"
    LIBRARY_OUTPUT_DIRECTORY "${PROJECT_SOURCE_DIR}/lib"
    RUNTIME_OUTPUT_DIRECTORY "${PROJECT_SOURCE_DIR}/bin"
)
```

其中：

- `ARCHIVE_OUTPUT_DIRECTORY`：静态库、Windows 导入库等归档产物输出目录；
- `LIBRARY_OUTPUT_DIRECTORY`：动态库、模块库等库产物输出目录；
- `RUNTIME_OUTPUT_DIRECTORY`：可执行程序，以及 Windows 下 DLL 等运行时产物输出目录。

如果只是学习基础语法，`EXECUTABLE_OUTPUT_PATH` 和 `LIBRARY_OUTPUT_PATH` 容易理解；如果写真实项目，更推荐使用上面的目标属性或

`CMAKE_RUNTIME_OUTPUT_DIRECTORY`、`CMAKE_LIBRARY_OUTPUT_DIRECTORY`、`CMAKE_ARCHIVE_OUTPUT_DIRECTORY`。

2.11 链接库文件

程序使用库函数时，不仅需要头文件通过编译，还需要在链接阶段链接对应库文件。

2.11.1 链接静态库

如果已有静态库：

```
lib
└─ libcalc.a
```

可以先指定库目录，再链接库：

```
cmake_minimum_required(VERSION 3.10)
project(TestCalc)

include_directories("${PROJECT_SOURCE_DIR}/include")
link_directories("${PROJECT_SOURCE_DIR}/lib")

file(GLOB SRC_LIST "${CMAKE_CURRENT_SOURCE_DIR}/src/*.cpp")

link_libraries(calc)
add_executable(app ${SRC_LIST})
```

`link_libraries(calc)` 中的 `calc` 对应 `libcalc.a`，通常可以省略 `lib` 前缀和 `.a` 后缀。

注意：`link_libraries()` 是目录级命令，会影响后续目标。现代 CMake 更推荐：

```
add_executable(app ${SRC_LIST})
target_link_libraries(app PRIVATE calc)
```

2.11.2 链接系统动态库

链接系统库常用 `target_link_libraries()`。以线程库为例，直接链接 `pthread` 在 Linux 下常见，但跨平台性不如 CMake 提供的 `Threads::Threads`

目标。

```
cmake_minimum_required(VERSION 3.10)
project(ThreadDemo)

find_package(Threads REQUIRED)

add_executable(app main.cpp)
target_link_libraries(app PRIVATE Threads::Threads)
```

`find_package(Threads REQUIRED)` 会让 CMake 根据当前平台找到合适的线程支持库或编译选项。Linux 下它可能对应 `pthread`，其他平台则可能不需要显式链接同名库。

2.11.3 链接第三方动态库

假设目录结构如下：

```
project
├─ CMakeLists.txt
├─ include
│   └─ head.h
├─ lib
│   └─ libcalc.so
└─ src
    └─ main.cpp
```

可以这样写：

```
cmake_minimum_required(VERSION 3.10)
project(TestCalc)

include_directories("${PROJECT_SOURCE_DIR}/include")
link_directories("${PROJECT_SOURCE_DIR}/lib")

add_executable(app src/main.cpp)
target_link_libraries(app PRIVATE calc)
```

注意：编译链接成功不代表运行时一定能找到动态库。Linux 程序启动后仍需要动态链接器能找到 `.so` 文件。常见解决方式包括：

- 把动态库放到系统库目录；
- 设置 `LD_LIBRARY_PATH`；
- 设置 `rpath`；
- 安装库文件到规范位置。

例如临时运行：

```
export LD_LIBRARY_PATH=$LD_LIBRARY_PATH:/path/to/project/lib
./app
```

2.12 日志输出

CMake 使用 `message()` 输出日志。

```
message([模式] "要输出的信息")
```

常见模式：

- 不写模式：重要消息；
- `STATUS`：普通状态消息；
- `WARNING`：警告，继续执行；
- `AUTHOR_WARNING`：开发者警告，继续执行；
- `SEND_ERROR`：错误，继续执行配置，但不生成构建系统；
- `FATAL_ERROR`：严重错误，立即终止。

示例：

```
message(STATUS "source dir: ${PROJECT_SOURCE_DIR}")
message(WARNING "this is a warning")
message(FATAL_ERROR "stop configure")
```

调试 `CMakeLists.txt` 时，`message(STATUS ...)` 很常用。

2.13 变量和列表操作

CMake 中的列表本质上是用分号分隔的字符串。`set(a b c)` 会形成一个列表。

2.13.1 使用 `set` 追加

```
set(SRC_1 main.cpp add.cpp)
set(SRC_2 sub.cpp div.cpp)

set(SRC_LIST ${SRC_1} ${SRC_2})
message(STATUS "src list: ${SRC_LIST}")
```

2.13.2 使用 `list(APPEND)` 追加

```
set(SRC_LIST main.cpp)
list(APPEND SRC_LIST add.cpp sub.cpp div.cpp)
message(STATUS "src list: ${SRC_LIST}")
```

2.13.3 移除列表元素

```
file(GLOB SRC_LIST "${PROJECT_SOURCE_DIR}/src/*.cpp")

# 移除 main.cpp。注意 file(GLOB) 得到的通常是完整路径。
list(REMOVE_ITEM SRC_LIST "${PROJECT_SOURCE_DIR}/src/main.cpp")
```

2.13.4 获取列表长度

```
set(NAMES Alice Bob Cindy)
list(LENGTH NAMES NAME_COUNT)
message(STATUS "count: ${NAME_COUNT}")
```

2.13.5 获取指定位置元素

```
set(NAMES Alice Bob Cindy)
list(GET NAMES 0 FIRST_NAME)
message(STATUS "first: ${FIRST_NAME}")
```

2.13.6 查找元素

```
set(NAMES Alice Bob Cindy)
list(FIND NAMES Bob BOB_INDEX)
message(STATUS "Bob index: ${BOB_INDEX}")
```

如果找不到，结果通常是 `-1`。

2.14 宏定义

CMake 可以给编译器添加宏定义。

2.14.1 add_definitions

旧式写法：

```
add_definitions(-DDEBUG)
```

C++ 中可以这样使用：

```
#ifdef DEBUG
std::cout << "debug mode\n";
#endif
```

也可以给宏赋值：

```
add_definitions(-DVERSION=3)
```

2.14.2 target_compile_definitions

现代 CMake 更推荐对具体目标添加宏：

```
add_executable(app main.cpp)
target_compile_definitions(app PRIVATE DEBUG)
```

作用范围：

- `PRIVATE`：只影响当前目标；

- `PUBLIC`：影响当前目标，也传播给依赖它的目标；
- `INTERFACE`：不影响当前目标，只传播给依赖它的目标。

示例：

```
add_library(calc STATIC add.cpp sub.cpp)
target_compile_definitions(calc PUBLIC USE_CALC_LIB)

add_executable(app main.cpp)
target_link_libraries(app PRIVATE calc)
```

2.15 常用预定义变量

CMake 提供了很多预定义变量，常用如下：

变量	含义
<code>PROJECT_SOURCE_DIR</code>	当前 <code>project()</code> 对应的源码根目录
<code>PROJECT_BINARY_DIR</code>	当前 <code>project()</code> 对应的构建目录
<code>CMAKE_CURRENT_SOURCE_DIR</code>	当前正在处理的 <code>CMakeLists.txt</code> 所在源码目录
<code>CMAKE_CURRENT_BINARY_DIR</code>	当前正在处理的 <code>CMakeLists.txt</code> 对应构建目录
<code>EXECUTABLE_OUTPUT_PATH</code>	可执行程序输出目录
<code>LIBRARY_OUTPUT_PATH</code>	库文件输出目录
<code>CMAKE_RUNTIME_OUTPUT_DIRECTORY</code>	初始化目标运行时产物输出目录
<code>CMAKE_LIBRARY_OUTPUT_DIRECTORY</code>	初始化目标库产物输出目录
<code>CMAKE_ARCHIVE_OUTPUT_DIRECTORY</code>	初始化目标归档产物输出目录
<code>PROJECT_NAME</code>	当前工程名
<code>CMAKE_BINARY_DIR</code>	顶层构建目录
<code>CMAKE_SOURCE_DIR</code>	顶层源码目录

变量	含义
CMAKE_CXX_STANDARD	C++ 标准版本

示例：

```
message(STATUS "project name: ${PROJECT_NAME}")
message(STATUS "source dir: ${PROJECT_SOURCE_DIR}")
message(STATUS "current dir: ${CMAKE_CURRENT_SOURCE_DIR}")
message(STATUS "build dir: ${CMAKE_BINARY_DIR}")
```

3 嵌套 CMake 工程

3.1 嵌套工程的目录关系

大型项目通常不会把所有源码都放到一个目录，而是拆成多个模块，每个模块有自己的 `CMakeLists.txt`。这种结构可以看成一棵树：

- 顶层 `CMakeLists.txt` 是根节点；
- 子目录中的 `CMakeLists.txt` 是子节点；
- 父节点可以把变量传给子节点；
- 子节点中普通变量默认只在当前目录及其子目录中有效。

示例目录：

```
project
├── CMakeLists.txt
├── calc
│   ├── CMakeLists.txt
│   └── add.cpp
├── sort
│   ├── CMakeLists.txt
│   └── sort.cpp
├── test_calc
│   ├── CMakeLists.txt
│   └── main.cpp
└── test_sort
    ├── CMakeLists.txt
    └── main.cpp
```

3.2 add_subdirectory

建立父子目录关系使用 `add_subdirectory()`。

```
add_subdirectory(source_dir [binary_dir] [EXCLUDE_FROM_ALL])
```

参数含义：

- `source_dir`：子目录路径，目录中通常要有 `CMakeLists.txt`；
- `binary_dir`：子目录对应的构建输出路径，一般可以省略；
- `EXCLUDE_FROM_ALL`：子目录目标默认不加入父目录的 `ALL` 目标，需要显式构建。

示例：

```
add_subdirectory(calc)
add_subdirectory(sort)
add_subdirectory(test_calc)
add_subdirectory(test_sort)
```

调用后，CMake 会立即处理子目录中的 `CMakeLists.txt`。

3.3 多目录工程示例

下面示例把 `calc` 和 `sort` 分别编译成库，再让测试程序链接这些库。

3.3.1 顶层 CMakeLists.txt

```
cmake_minimum_required(VERSION 3.10)
project(ModularDemo)

set(CMAKE_CXX_STANDARD 17)
set(CMAKE_CXX_STANDARD_REQUIRED ON)

set(LIB_PATH "${CMAKE_CURRENT_SOURCE_DIR}/lib")
set(EXEC_PATH "${CMAKE_CURRENT_SOURCE_DIR}/bin")
set(HEAD_PATH "${CMAKE_CURRENT_SOURCE_DIR}/include")

set(CALC_LIB calc)
set(SORT_LIB sort)
set(TEST_CALC test_calc)
set(TEST_SORT test_sort)

add_subdirectory(calc)
add_subdirectory(sort)
add_subdirectory(test_calc)
add_subdirectory(test_sort)
```

这个文件主要做两件事：

- 定义各子目录共用的变量；
- 添加子目录，建立构建关系。

3.3.2 calc/CMakeLists.txt

```
file(GLOB SRC_LIST "${CMAKE_CURRENT_SOURCE_DIR}/*.cpp")

include_directories("${HEAD_PATH}")
set(LIBRARY_OUTPUT_PATH "${LIB_PATH}")

add_library(${CALC_LIB} STATIC ${SRC_LIST})
```

3.3.3 sort/CMakeLists.txt

```
file(GLOB SRC_LIST "${CMAKE_CURRENT_SOURCE_DIR}/*.cpp")

include_directories("${HEAD_PATH}")
set(LIBRARY_OUTPUT_PATH "${LIB_PATH}")

add_library(${SORT_LIB} STATIC ${SRC_LIST})
```

3.3.4 test_calc/CMakeLists.txt

```
file(GLOB SRC_LIST "${CMAKE_CURRENT_SOURCE_DIR}/*.cpp")

include_directories("${HEAD_PATH}")
link_directories("${LIB_PATH}")
set(EXECUTABLE_OUTPUT_PATH "${EXEC_PATH}")

add_executable(${TEST_CALC} ${SRC_LIST})
target_link_libraries(${TEST_CALC} PRIVATE ${CALC_LIB})
```

3.3.5 test_sort/CMakeLists.txt

```
file(GLOB SRC_LIST "${CMAKE_CURRENT_SOURCE_DIR}/*.cpp")

include_directories("${HEAD_PATH}")
link_directories("${LIB_PATH}")
set(EXECUTABLE_OUTPUT_PATH "${EXEC_PATH}")

add_executable(${TEST_SORT} ${SRC_LIST})
target_link_libraries(${TEST_SORT} PRIVATE ${SORT_LIB})
```

这样构建后，库文件会输出到 `lib`，测试程序会输出到 `bin`。

4 CMake 流程控制

CMakeLists.txt 中可以进行条件判断和循环，语法类似脚本语言。

4.1 条件判断 if

基本格式：

```
if(<condition>)  
    <commands>  
elseif(<condition>)  
    <commands>  
else()  
    <commands>  
endif()
```

示例：

```
set(BUILD_TEST ON)  
  
if(BUILD_TEST)  
    message(STATUS "build test enabled")  
else()  
    message(STATUS "build test disabled")  
endif()
```

`if()` 和 `endif()` 必须成对出现，`elseif()` 和 `else()` 可选。

4.2 基本表达式

`if(<expression>)` 中的表达式可以是常量、变量或字符串。

常见真值常量：

- 1
- ON
- YES
- TRUE
- Y
- 非零数值

常见假值常量：

- `0`
- `OFF`
- `NO`
- `FALSE`
- `N`
- `IGNORE`
- `NOTFOUND`
- 空字符串
- 以 `-NOTFOUND` 结尾的值

需要特别注意：CMake 的 `if()` 并不是“任意非空字符串都为真”。未加引号的参数会先尝试按变量名解释；如果变量已定义且值不是假值常量，则为真。如果写成带引号的普通字符串，除非字符串内容本身是 `ON`、`TRUE`、非零数字等真值常量，否则通常为假。

示例：

```
set(ENABLE_LOG ON)

if(ENABLE_LOG)
    message(STATUS "log enabled")
endif()

if("hello")
    message(STATUS "this line usually will not be printed")
endif()
```

4.3 逻辑判断

CMake 支持 `NOT`、`AND`、`OR`。

```
if(NOT ENABLE_TEST)
    message(STATUS "test disabled")
endif()
```

```
if(ENABLE_TEST AND ENABLE_LOG)
    message(STATUS "test and log enabled")
endif()
```

```
if(WIN32 OR UNIX)
    message(STATUS "known platform")
endif()
```

复杂条件建议加括号增强可读性：

```
if((CMAKE_BUILD_TYPE STREQUAL "Debug") AND ENABLE_LOG)
    message(STATUS "debug log enabled")
endif()
```

4.4 数值比较

常用数值比较：

```
if(A LESS B)
if(A GREATER B)
if(A EQUAL B)
if(A LESS_EQUAL B)
if(A GREATER_EQUAL B)
```

示例：

```
set(VERSION_CODE 3)

if(VERSION_CODE GREATER_EQUAL 2)
    message(STATUS "version is ok")
endif()
```

4.5 字符串比较

常用字符串比较：

```
if(A STREQUAL B)
if(A STRLESS B)
if(A STRGREATER B)
if(A STRLESS_EQUAL B)
if(A STRGREATER_EQUAL B)
```

示例：

```
set(BUILD_MODE "debug")

if(BUILD_MODE STREQUAL "debug")
    add_definitions(-DDEBUG)
endif()
```

字符串匹配：

```
set(NAME "Alice")

if(NAME MATCHES "^[A-Z]")
    message(STATUS "name starts with uppercase letter")
endif()
```

4.6 文件和路径判断

4.6.1 判断是否存在

```
if(EXISTS "${PROJECT_SOURCE_DIR}/config.json")
    message(STATUS "config exists")
endif()
```

`EXISTS` 可以判断文件或目录是否存在。

4.6.2 判断是否为目录

```
if(IS_DIRECTORY "${PROJECT_SOURCE_DIR}/include")
    message(STATUS "include is a directory")
endif()
```

4.6.3 判断是否为软链接

```
if(IS_SYMLINK "${PROJECT_SOURCE_DIR}/latest")
    message(STATUS "latest is a symlink")
endif()
```

4.6.4 判断是否为绝对路径

```
if(IS_ABSOLUTE "${PROJECT_SOURCE_DIR}")
    message(STATUS "absolute path")
endif()
```

Linux 绝对路径通常从 `/` 开始，Windows 绝对路径通常从盘符开始。

4.7 列表和路径比较

4.7.1 判断元素是否在列表中

`IN_LIST` 要求 CMake 版本不低于 3.3。

```
cmake_minimum_required(VERSION 3.10)
project(ListDemo)

set(MODULES calc sort network)

if("calc" IN_LIST MODULES)
    message(STATUS "calc module enabled")
endif()
```

4.7.2 比较路径是否相等

`PATH_EQUAL` 用于路径比较，能处理路径分隔符重复等情况。该功能要求较新的 CMake 版本。

```
cmake_minimum_required(VERSION 3.24)
project(PathDemo)

if("/home//user///project" PATH_EQUAL "/home/user/project")
    message(STATUS "path equal")
else()
    message(STATUS "path not equal")
endif()
```

如果使用字符串比较：

```
if("/home//user///project" STREQUAL "/home/user/project")
    message(STATUS "string equal")
else()
    message(STATUS "string not equal")
endif()
```

上面两个路径在字符串层面不相等，但路径语义上可以认为相等，所以路径比较应优先使用 `PATH_EQUAL`。

4.8 循环 foreach

`foreach()` 用于遍历列表或范围。

4.8.1 遍历普通列表

```
set(MODULES calc sort network)

foreach(module ${MODULES})
    message(STATUS "module: ${module}")
endforeach()
```

4.8.2 遍历显式元素

```
foreach(name Alice Bob Cindy)
    message(STATUS "name: ${name}")
endforeach()
```

4.8.3 RANGE

```
foreach(i RANGE 1 5)
  message(STATUS "i = ${i}")
endforeach()
```

指定步长：

```
foreach(i RANGE 0 10 2)
  message(STATUS "i = ${i}")
endforeach()
```

4.8.4 配合源文件列表

```
file(GLOB SRC_LIST "${PROJECT_SOURCE_DIR}/src/*.cpp")

foreach(src ${SRC_LIST})
  message(STATUS "source: ${src}")
endforeach()
```

4.9 循环 while

`while()` 会在条件为真时重复执行命令。

```
set(i 0)

while(i LESS 5)
  message(STATUS "i = ${i}")
  math(EXPR i "${i} + 1")
endwhile()
```

`math(EXPR ...)` 用于计算整数表达式。

示例：生成多个宏定义。

```
set(i 1)

while(i LESS_EQUAL 3)
    add_definitions(-DLEVEL_${i})
    math(EXPR i "${i} + 1")
endwhile()
```

在实际工程中，`foreach()` 比 `while()` 更常用，因为 CMake 中大量数据都以列表形式出现。

5 综合示例

下面给出一个完整小工程，覆盖源文件搜索、头文件包含、静态库生成、可执行程序生成、库链接、输出目录和子目录管理。

目录结构：

```
demo
├── CMakeLists.txt
├── include
│   └── calc.h
├── calc
│   ├── CMakeLists.txt
│   ├── add.cpp
│   └── sub.cpp
└── app
    ├── CMakeLists.txt
    └── main.cpp
```

顶层 `CMakeLists.txt`：

```
cmake_minimum_required(VERSION 3.10)
project(CMakeDemo)

set(CMAKE_CXX_STANDARD 17)
set(CMAKE_CXX_STANDARD_REQUIRED ON)

set(HEAD_PATH "${CMAKE_CURRENT_SOURCE_DIR}/include")
set(LIB_PATH "${CMAKE_CURRENT_SOURCE_DIR}/lib")
set(EXEC_PATH "${CMAKE_CURRENT_SOURCE_DIR}/bin")
set(CALC_LIB calc)
set(APP_NAME app)

add_subdirectory(calc)
add_subdirectory(app)
```

calc/CMakeLists.txt :

```
file(GLOB SRC_LIST "${CMAKE_CURRENT_SOURCE_DIR}/*.cpp")

include_directories("${HEAD_PATH}")
set(LIBRARY_OUTPUT_PATH "${LIB_PATH}")

add_library(${CALC_LIB} STATIC ${SRC_LIST})
```

app/CMakeLists.txt :

```
include_directories("${HEAD_PATH}")
link_directories("${LIB_PATH}")
set(EXECUTABLE_OUTPUT_PATH "${EXEC_PATH}")

add_executable(${APP_NAME} main.cpp)
target_link_libraries(${APP_NAME} PRIVATE ${CALC_LIB})
```

构建：

```
mkdir build
cd build
cmake ..
cmake --build .
```

构建结果：

```
demo
├─ bin
│   └─ app
└─ lib
    └─ libcalc.a
```

这个示例体现了 CMake 常用组织方式：

- 顶层负责全局变量和子目录组织；
- 子模块负责生成库；
- 应用模块负责生成可执行程序并链接库；
- 头文件目录、库输出目录、可执行程序输出目录统一管理。

掌握这些内容后，就可以阅读和维护中小型 C/C++ 项目的 CMake 构建脚本。